

Competitive Programming

Section 14 Editorial: Core Patterns Mixed Contest

Competitive Programming Course

2026-08-02

Table of contents

1	Category coverage	7
2	Editorial overview	8
3	A. Complete Window	9
3.1	Restatement	9
3.2	Constraints and traps	9
3.3	Observations	9
3.4	Sample walk-through	9
3.5	Algorithm	10
3.6	Correctness proof	10
3.7	Complexity	10
3.8	Full C++ solution	11
3.9	Full Python solution	12
3.10	Tests that matter	13
4	B. Minimum Split Limit	14
4.1	Restatement	14
4.2	Constraints and traps	14
4.3	Observations	14
4.4	Sample walk-through	14
4.5	Algorithm	15
4.6	Correctness proof	15
4.7	Complexity	16
4.8	Full C++ solution	16
4.9	Full Python solution	17
4.10	Tests that matter	18
5	C. Active Colors	19
5.1	Restatement	19
5.2	Constraints and traps	19
5.3	Observations	19
5.4	Sample walk-through	20
5.5	Algorithm	20
5.6	Correctness proof	20
5.7	Complexity	21
5.8	Full C++ solution	21
5.9	Full Python solution	23
5.10	Tests that matter	24
6	Stress testing	25
7	What to carry forward	26

8	Problem D: First Free Slot	27
8.1	Restatement	27
8.2	Constraints and traps	27
8.3	Observation and walk-through	27
8.4	Algorithm	27
8.5	Correctness proof	28
8.6	Complexity	28
8.7	Full C++ solution	28
8.8	Full Python solution	28
8.9	Tests that matter	29
9	E. Smallest Distinct Subsequence	30
9.1	Restatement and recognition	30
9.2	Maintained information	30
9.3	Algorithm	30
9.4	Correctness proof	31
9.5	Complexity	31
9.6	Common wrong answers	31
9.7	Tests that matter	31
9.8	Full C++ solution	32
9.9	Full Python solution	32
10	F. Adjacent Cancellation	34
10.1	Recognition and stack state	34
10.2	Correctness proof	34
10.3	Complexity, mistakes, and tests	34
10.4	Full C++ solution	35
10.5	Full Python solution	35
11	G. Next Smaller Distance	36
11.1	From unresolved queries to a monotonic stack	36
11.2	Correctness proof	36
11.3	Complexity and common mistakes	36
11.4	Full C++ solution	36
11.5	Full Python solution	37
12	H. Smallest Fixed-Length Subsequence	38
12.1	Greedy interpretation	38
12.2	Correctness proof	38
12.3	Complexity, pitfalls, and tests	38
12.4	Full C++ solution	39
12.5	Full Python solution	39
13	I. Wildcard Parentheses	40
13.1	How to discover the compression	40
13.2	Feasible-balance interval	41
13.3	Correctness proof	41
13.4	Complexity, mistakes, and tests	42
13.5	Full C++ solution	42
13.6	Full Python solution	43
14	J. Interval Point Cover	44
14.1	Greedy choice	44

14.2	Exchange proof	44
14.3	Complexity and tests	44
14.4	Full C++ solution	44
14.5	Full Python solution	45
15	K. Deadline Selection	46
15.1	Greedy subset maintenance	46
15.2	Correctness proof	46
15.3	Complexity, pitfalls, and tests	46
15.4	Full C++ solution	47
15.5	Full Python solution	47
16	L. Smallest After Deletions	49
16.1	From H to an explicit deletion budget	49
16.2	Why unused deletions come from the end	49
16.3	Correctness proof	49
16.4	Complexity, pitfalls, and tests	50
16.5	Full C++ solution	50
16.6	Full Python solution	50
17	M. Required Letter Subsequence	52
17.1	Why H's single feasibility check is no longer enough	52
17.2	Deciding whether to push	52
17.3	Correctness proof	53
17.4	Complexity, pitfalls, and tests	53
17.5	Full C++ solution	53
17.6	Full Python solution	54
18	N. Wildcard Choice Map	56
18.1	Ask about one position without fixing all the others	56
18.2	Backward balances	56
18.3	Test each replacement by intersection	57
18.4	Correctness proof	57
18.5	Complexity, pitfalls, and tests	57
18.6	Full C++ solution	58
18.7	Full Python solution	59
19	O. Minimum Parenthesis Depth	61
19.1	Turn optimization into a feasibility question	61
19.2	Why the compressed interval remains exact	61
19.3	Worked example	62
19.4	Complexity, pitfalls, and tests	62
19.5	Full C++ solution	62
19.6	Full Python solution	63
20	P. Common Deadline	65
20.1	Isolate the exchange behind deadline selection	65
20.2	Exchange proof	65
20.3	Connection to K	65
20.4	Complexity, pitfalls, and tests	66
20.5	Full C++ solution	66
20.6	Full Python solution	66

21 Q. Event Attendance	68
21.1 Two orders matter	68
21.2 Exchange proof	68
21.3 Worked example	68
21.4 Complexity, pitfalls, and tests	69
21.5 Full C++ solution	69
21.6 Full Python solution	70
22 R. Coverage Patches	71
22.1 Discover the state before the choice	71
22.2 Why the forced patch is covered + 1	71
22.3 Correctness proof	72
22.4 Complexity, pitfalls, and tests	72
22.5 Full C++ solution	72
22.6 Full Python solution	73
23 S. Advantage Assignment	74
23.1 Greedy matching question	74
23.2 Exchange proof	74
23.3 Why sorting both arrays is legitimate	74
23.4 Complexity, pitfalls, and tests	75
23.5 Full C++ solution	75
23.6 Full Python solution	75
24 T. Minimum Refueling	77
24.1 Postpone the decision	77
24.2 Why a retroactive stop is valid	77
24.3 Exchange proof	77
24.4 Sample trace	77
24.5 Complexity, pitfalls, and tests	78
24.6 Full C++ solution	78
24.7 Full Python solution	79
25 U. Circular Fuel Start	80
25.1 Two separate questions	80
25.2 Failed-block proof	80
25.3 Why the final candidate completes the circle	80
25.4 Complexity, pitfalls, and tests	81
25.5 Full C++ solution	81
25.6 Full Python solution	81
26 V. Smallest Separated Rearrangement	83
26.1 Lexicographic construction needs a completion test	83
26.2 Making the test fast	83
26.3 Correctness proof	83
26.4 Complexity, pitfalls, and tests	84
26.5 Full C++ solution	84
26.6 Full Python solution	85
27 W. Consecutive Grouping	87
27.1 Obligations before opportunities	87
27.2 Why extending first is forced	87
27.3 Why starting with three values is safe	87

27.4	Complexity, pitfalls, and tests	88
27.5	Full C++ solution	88
27.6	Full Python solution	89
28	X. Subarray Minimum Total	90
28.1	Start from subarrays ending at one position	90
28.2	Why merging equal values is convenient	90
28.3	Correctness proof	90
28.4	Complexity, pitfalls, and tests	91
28.5	Full C++ solution	91
28.6	Full Python solution	92
29	Y. Maximum Width Ramp	93
29.1	Eliminate dominated left endpoints	93
29.2	Why scan right endpoints backward	93
29.3	Correctness proof	93
29.4	Complexity, pitfalls, and tests	93
29.5	Full C++ solution	94
29.6	Full Python solution	94
30	Z. Smallest Alternating Partition	96
30.1	Separate feasibility from label choice	96
30.2	Correctness proof	96
30.3	Complexity, pitfalls, and tests	96
30.4	Full C++ solution	97
30.5	Full Python solution	97

1 Category coverage

Category	Coverage problem
binary search on the answer	B. Minimum Split Limit
greedy exchange	D. First Free Slot
constructive greedy and invariants	E. Smallest Distinct Subsequence
intervals and sweep line	C. Active Colors; D. First Free Slot
stacks, queues, and dequeues	E. Smallest Distinct Subsequence
offline processing	C. Active Colors

Problems F–Z form four additional stack/greedy training ladders. They include statements, stubs, reference solutions, fixed tests, and randomized tests, and this section’s checker includes them.

2 Editorial overview

This mixed contest is about choosing tools.

The intended pattern decisions are:

Problem	Pattern
A. Complete Window	sliding window with frequencies
B. Minimum Split Limit	binary search on answer
C. Active Colors	sweep line with answer restoration
D. First Free Slot	sorted greedy interval placement
E. Smallest Distinct Subsequence	greedy monotonic stack with suffix feasibility
F. Adjacent Cancellation	stack reduction
G. Next Smaller Distance	monotonic stack
H. Smallest Fixed-Length Subsequence	lexicographic monotonic stack
I. Wildcard Parentheses	greedy feasible-balance interval
J. Interval Point Cover	earliest-finish exchange greedy
K. Deadline Selection	deadline order plus maximum heap
L. Smallest After Deletions	deletion-budget monotonic stack
M. Required Letter Subsequence	monotonic stack with dual feasibility
N. Wildcard Choice Map	prefix/suffix balance-interval intersection
O. Minimum Parenthesis Depth	balance interval plus binary search
P. Common Deadline	shortest-duration exchange greedy
Q. Event Attendance	day sweep plus minimum heap
R. Coverage Patches	representable-prefix invariant
S. Advantage Assignment	sorted matching exchange
T. Minimum Refueling	retroactive maximum-heap greedy
U. Circular Fuel Start	failed-prefix candidate elimination
V. Smallest Separated Rearrangement	lexicographic feasibility greedy
W. Consecutive Grouping	extend-before-start obligation greedy
X. Subarray Minimum Total	monotonic stack of suffix-minimum contributions
Y. Maximum Width Ramp	decreasing candidate stack plus reverse scan
Z. Smallest Alternating Partition	forced reuse plus minimum group id

The editorial explains both the algorithm and the recognition step.

3 A. Complete Window

3.1 Restatement

Find the shortest contiguous subarray containing every required label from 1 through m at least once. Values outside this range are irrelevant.

3.2 Constraints and traps

The sum of n is large, so checking every subarray is too slow.

Common traps:

- counting irrelevant labels as useful;
- stopping after finding the first valid window;
- not shrinking the left side after a window becomes valid.

3.3 Observations

The statement asks for a shortest contiguous subarray and the validity condition is based on counts inside the current window. This suggests sliding window.

Maintain:

```
freq[x] = occurrences of required label x inside the window
have = number of required labels whose frequency is positive
```

When `have == m`, the current window is valid. Shrink it from the left to find the shortest valid window ending at the current right endpoint.

3.4 Sample walk-through

Array:

```
5 1 2 9 3 2 1 3
m = 3
```

The window `[1,2,9,3]` is valid with length 4. After shrinking away the left 1, it becomes invalid.

Later, `[3,2,1]` is valid with length 3, which is optimal.

3.5 Algorithm

```
left = 0
have = 0
best = infinity
freq = zeros for labels 1..m

for right in 0..n-1:
    add a[right] if it is required
    while have == m:
        best = min(best, right-left+1)
        remove a[left] if it is required
        left += 1

print best or -1
```

3.6 Correctness proof

Lemma 1. At all times, `freq` and `have` correctly describe the current window `[left, right]`.

Proof. The algorithm increments a required label's count when the right endpoint enters the window and decrements it when the left endpoint leaves. `have` changes exactly when a required label's count moves between zero and one. $\square$

Lemma 2. For each fixed `right`, the algorithm checks every minimal valid window ending at `right`.

Proof. Once the window is valid, the algorithm repeatedly removes the left endpoint and records the length before each removal. It stops exactly when the window becomes invalid. Therefore the last valid window before stopping is the shortest valid window ending at `right`, and all longer valid windows during that shrinking process were also considered. $\square$

Theorem. The algorithm returns the length of the shortest valid subarray.

Proof. Every valid subarray has some right endpoint. By Lemma 2, when that right endpoint is processed, the algorithm considers the shortest valid window ending there, whose length is no greater than the chosen subarray. Taking the minimum over all right endpoints gives the global optimum. $\square$

3.7 Complexity

Each endpoint moves right at most `n` times.

```
Time:  $O(n)$ 
Memory:  $O(m)$ 
```

3.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int solve_case(const vector<int> &a, int m) {
    vector<int> freq(m + 1, 0);
    int have = 0;
    int best = (int)a.size() + 1;
    int left = 0;
    for (int right = 0; right < (int)a.size(); ++right) {
        int value = a[right];
        if (1 <= value && value <= m) {
            if (freq[value] == 0) ++have;
            ++freq[value];
        }
        while (have == m) {
            best = min(best, right - left + 1);
            int old = a[left];
            if (1 <= old && old <= m) {
                --freq[old];
                if (freq[old] == 0) --have;
            }
            ++left;
        }
    }
    return best == (int)a.size() + 1 ? -1 : best;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, m;
        cin >> n >> m;
        vector<int> a(n);
        for (int &x : a) cin >> x;
        cout << solve_case(a, m) << '\n';
    }

    return 0;
}
```

3.9 Full Python solution

```
import sys

def solve_case(a: list[int], m: int) -> int:
    freq = [0] * (m + 1)
    have = 0
    best = len(a) + 1
    left = 0
    for right, value in enumerate(a):
        if 1 <= value <= m:
            if freq[value] == 0:
                have += 1
            freq[value] += 1
        while have == m:
            best = min(best, right - left + 1)
            old = a[left]
            if 1 <= old <= m:
                freq[old] -= 1
                if freq[old] == 0:
                    have -= 1
            left += 1
    return -1 if best == len(a) + 1 else best

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        m = data[idx + 1]
        idx += 2
        a = data[idx:idx + n]
        idx += n
        out.append(str(solve_case(a, m)))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()
```

3.10 Tests that matter

```
valid window at the beginning  
valid window at the end  
irrelevant values inside the best window  
missing required label -> -1
```

4 B. Minimum Split Limit

4.1 Restatement

Split an array of positive integers into at most d nonempty contiguous groups. Minimize the largest group sum.

4.2 Constraints and traps

The input satisfies $1 \leq d \leq n$, $1 \leq a_i \leq 10^9$, and the sum of n over all test cases is at most 200000. Individual elements fit in 32 bits, but $\text{sum}(a)$ and the answer may reach $2 * 10^{14}$, so C++ must use `long long` for group sums and binary-search bounds.

The array order cannot change. Sorting the values is invalid.

Common traps:

- binary searching below $\max(a)$;
- using exactly d groups when the statement says at most d ;
- forgetting that all values are positive, which makes the greedy check valid.

4.3 Observations

Let C be a candidate maximum group sum.

Predicate:

```
feasible(C) = we can split into at most d groups with every group sum <= C
```

If C is feasible, any larger value is also feasible. So we binary search the minimum feasible C .

For a fixed C , greedily fill each group as much as possible before starting a new group. This minimizes the number of groups needed for that C .

4.4 Sample walk-through

Array:

```
7 2 5 10 8
d = 2
```

Check $C = 18$:

```
group 1: 7+2+5 = 14
next 10 would exceed 18
group 2: 10+8 = 18
groups = 2, feasible
```

Check $C = 17$:

```
group 1: 7+2+5 = 14
group 2: 10
group 3: 8
groups = 3, not feasible
```

Answer: 18.

4.5 Algorithm

```
lo = max(a)
hi = sum(a)
while lo < hi:
    mid = (lo + hi) // 2
    if groups_needed(mid) <= d:
        hi = mid
    else:
        lo = mid + 1
print lo
```

4.6 Correctness proof

Lemma 1. For a fixed capacity C , the greedy scan uses the minimum possible number of groups.

Proof. The greedy scan ends a group only when the next element would exceed C . Any valid solution's first group cannot extend beyond that point, because including that next element would also exceed C . Therefore the greedy first group is at least as long as the first group of some optimal split. Repeating the argument on the remaining suffix proves the greedy scan uses no more groups than any other valid split. $\square$

Lemma 2. The feasibility predicate is monotone.

Proof. If capacity C allows a valid split, then any capacity $C' \geq C$ allows the same split. $\square$

Theorem. The binary search returns the minimum possible maximum group sum.

Proof. The lower bound $\max(a)$ is necessary because every element must fit in some group. The upper bound $\sum(a)$ is feasible by taking one group. By Lemma 2, feasible capacities form a suffix of the search range. The binary search maintains the minimum feasible capacity inside $[lo, hi]$ and terminates when $lo == hi$, which is that minimum. $\square$

4.7 Complexity

Time: $O(n \log \text{sum}(a))$
Memory: $O(1)$ besides input

4.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int groups_needed(const vector<long long> &a, long long limit) {
    int groups = 1;
    long long current = 0;
    for (long long x : a) {
        if (current + x <= limit) {
            current += x;
        } else {
            ++groups;
            current = x;
        }
    }
    return groups;
}

long long solve_case(const vector<long long> &a, int d) {
    long long lo = *max_element(a.begin(), a.end());
    long long hi = accumulate(a.begin(), a.end(), 0LL);
    while (lo < hi) {
        long long mid = lo + (hi - lo) / 2;
        if (groups_needed(a, mid) <= d) {
            hi = mid;
        } else {
            lo = mid + 1;
        }
    }
    return lo;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, d;
        cin >> n >> d;
```

```

        vector<long long> a(n);
        for (long long &x : a) cin >> x;
        cout << solve_case(a, d) << '\n';
    }

    return 0;
}

```

4.9 Full Python solution

```

import sys

def groups_needed(a: list[int], limit: int) -> int:
    groups = 1
    current = 0
    for x in a:
        if current + x <= limit:
            current += x
        else:
            groups += 1
            current = x
    return groups

def solve_case(a: list[int], d: int) -> int:
    lo = max(a)
    hi = sum(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if groups_needed(a, mid) <= d:
            hi = mid
        else:
            lo = mid + 1
    return lo

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        d = data[idx + 1]
        idx += 2

```

```
        a = data[idx:idx + n]
        idx += n
        out.append(str(solve_case(a, d)))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()
```

4.10 Tests that matter

```
d = 1 -> answer is sum(a)
d = n -> answer is max(a)
large values -> 64-bit sums
two huge endpoint values with small middle values
```

5 C. Active Colors

5.1 Restatement

Given colored half-open intervals $[l, r)$ and query points, output for each query the number of distinct colors with at least one active interval at that point.

5.2 Constraints and traps

Checking every interval for every query is too slow.

Common traps:

- treating $[l, r)$ as closed;
- counting intervals instead of distinct colors;
- printing answers in event-sorted order;
- losing colors with multiple active intervals.

5.3 Observations

This is a sweep-line problem with a frequency map.

Events:

```
kind 0: interval end at r
kind 1: interval start at l
kind 2: query at x
```

Tie order:

```
end -> start -> query
```

This models half-open intervals because ending intervals are removed at x before the query, while starting intervals are added before the query.

Maintain:

```
freq[color] = number of active intervals of that color
distinct = number of colors with positive frequency
```

5.4 Sample walk-through

Intervals:

```
[1,5) color 10
[2,6) color 20
[4,7) color 10
[7,9) color 30
```

At $x = 4$, active intervals have colors 10,20,10, so the distinct color count is 2.

At $x = 7$, the $[4,7)$ interval has ended and $[7,9)$ has started, so the active color set is $\{30\}$.

5.5 Algorithm

```
events = []
for each interval:
    add (r, end, color)
    add (l, start, color)
for each query with id:
    add (x, query, id)

sort events by (coordinate, kind)

freq = empty map
distinct = 0
ans = array of q

for event:
    if end:
        decrease color frequency
        if it becomes zero, distinct -= 1
    if start:
        if color frequency was zero, distinct += 1
        increase color frequency
    if query:
        ans[id] = distinct
```

5.6 Correctness proof

Lemma 1. When a query at coordinate x is processed, $\text{freq}[\text{color}]$ equals the number of active intervals of that color at x .

Proof. Any interval ending at x has event kind 0, so it is removed before the query. This matches $[l,r)$, which excludes r . Any interval starting at x has event kind 1, so it is added before the query. This matches $[l,r)$, which includes l . Intervals with smaller endpoints have already been processed, and intervals with larger endpoints have not. $\square$

Lemma 2. At every point in the sweep, `distinct` equals the number of colors with positive active frequency.

Proof. On a start event, `distinct` increases exactly when the previous frequency was zero. On an end event, it decreases exactly when the frequency becomes zero. Other changes do not affect whether the color is present. $\square$

Theorem. The algorithm returns the correct answer for every query in original order.

Proof. By Lemma 1, the frequency map is correct at each query coordinate. By Lemma 2, `distinct` is exactly the number of active colors. The query's original id stores the answer in the correct output slot. $\square$

5.7 Complexity

There are $2n + q$ events.

```
Time:  $O((n+q) \log(n+q))$   
Memory:  $O(n+q)$ 
```

5.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Event {
    long long x;
    int kind;
    long long color;
    int idx;
};

vector<int> solve_case(
    const vector<tuple<long long, long long, long long>> &intervals,
    const vector<long long> &queries
) {
    vector<Event> events;
    events.reserve(intervals.size() * 2 + queries.size());
    for (auto [l, r, color] : intervals) {
        events.push_back({r, 0, color, -1});
        events.push_back({l, 1, color, -1});
    }
    for (int i = 0; i < (int)queries.size(); ++i) {
        events.push_back({queries[i], 2, 0, i});
    }
    sort(events.begin(), events.end(), [](const Event &a, const Event &b) {
        if (a.x != b.x) return a.x < b.x;
        return a.kind < b.kind;
    });
};
```

```

unordered_map<long long, int> freq;
int distinct = 0;
vector<int> ans(queries.size());
for (const Event &event : events) {
    if (event.kind == 0) {
        int count = freq[event.color] - 1;
        if (count == 0) {
            --distinct;
            freq.erase(event.color);
        } else {
            freq[event.color] = count;
        }
    } else if (event.kind == 1) {
        int count = freq[event.color];
        if (count == 0) ++distinct;
        freq[event.color] = count + 1;
    } else {
        ans[event.idx] = distinct;
    }
}
return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, q;
        cin >> n >> q;
        vector<tuple<long long, long long, long long>> intervals;
        intervals.reserve(n);
        for (int i = 0; i < n; ++i) {
            long long l, r, color;
            cin >> l >> r >> color;
            intervals.push_back({l, r, color});
        }
        vector<long long> queries(q);
        for (long long &x : queries) cin >> x;
        vector<int> ans = solve_case(intervals, queries);
        for (int i = 0; i < q; ++i) {
            if (i) cout << ' ';
            cout << ans[i];
        }
        cout << '\n';
    }

    return 0;
}

```

```
}
```

5.9 Full Python solution

```
import sys

def solve_case(intervals: list[tuple[int, int, int]], queries: list[int]) -> list[int]:
    events: list[tuple[int, int, int, int]] = []
    for l, r, color in intervals:
        events.append((r, 0, color, -1))
        events.append((l, 1, color, -1))
    for idx, x in enumerate(queries):
        events.append((x, 2, 0, idx))
    events.sort()

    freq: dict[int, int] = {}
    distinct = 0
    ans = [0] * len(queries)
    for _, kind, color, idx in events:
        if kind == 0:
            count = freq[color] - 1
            if count == 0:
                distinct -= 1
                del freq[color]
            else:
                freq[color] = count
        elif kind == 1:
            count = freq.get(color, 0)
            if count == 0:
                distinct += 1
            freq[color] = count + 1
        else:
            ans[idx] = distinct
    return ans

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        q = data[idx + 1]
        idx += 2
```

```

intervals: list[tuple[int, int, int]] = []
for _ in range(n):
    l = data[idx]
    r = data[idx + 1]
    color = data[idx + 2]
    idx += 3
    intervals.append((l, r, color))
queries = data[idx:idx + q]
idx += q
out.append(" ".join(map(str, solve_case(intervals, queries))))
sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

5.10 Tests that matter

```

query exactly at l
query exactly at r
multiple active intervals with same color
duplicate query points
queries before all intervals and after all intervals

```

6 Stress testing

For Problem A, brute force every subarray.

For Problem B, brute force all split positions or use a small dynamic program.

For Problem C, answer every query by scanning all intervals and building a set of active colors.

The local generators use exactly these oracles.

7 What to carry forward

Core-pattern contests are won by disciplined recognition:

- contiguous and maintainable -> sliding window;
- monotone feasibility -> binary search;
- endpoints and active state -> sweep line;
- sorted queries -> offline processing.

The next phase starts graphs. The same habit remains: read constraints, model the state, choose the tool, then prove the invariant.

8 Problem D: First Free Slot

8.1 Restatement

Given closed busy intervals within $0..limit$ and a required integer duration d , find the smallest start $s \geq 0$ whose entire interval $[s, s+d-1]$ is not busy. Return -1 if no such start exists.

8.2 Constraints and traps

The total number of intervals is at most 200000, so testing every possible start is impossible. Intervals are closed: if one ends at r , the earliest time after it is $r+1$, not r . Input intervals may overlap, nest, or touch, and $1 \leq r$ is guaranteed. Use `long long` for $s+d-1$.

8.3 Observation and walk-through

Sort closed busy intervals and maintain `cur`, the earliest time not already blocked by processed intervals. If $cur+d-1 < l$, the gap before the next interval fits and `cur` is the answer. Otherwise merge the obstacle by setting $cur = \max(cur, r+1)$. After all intervals, accept only when $cur+d-1 \leq limit$.

For busy intervals $[1,2], [5,7], [7,9]$, duration three, and limit twelve, start `cur=0`. The initial candidate $[0,2]$ overlaps $[1,2]$, so move to 3. $[3,5]$ overlaps $[5,7]$, so move to 8; $[8,10]$ overlaps $[7,9]$, so move to 10. Finally $[10,12]$ is free, so the answer is 10.

8.4 Algorithm

```
sort intervals by left endpoint
cur = 0
for [l, r] in sorted intervals:
    if cur + d - 1 < l: return cur
    cur = max(cur, r + 1)
return cur if cur + d - 1 <= limit else -1
```

8.5 Correctness proof

The invariant is that no start below `cur` can still begin a legal slot after the processed intervals. Sorting makes every future interval start no earlier than the current one, so a fitting gap can be returned immediately. Each interval either blocks the current candidate and advances `cur` past its right endpoint, or leaves a gap large enough to return the smallest possible start. After all intervals, the final limit test accepts exactly the remaining legal candidate. Thus the algorithm returns the earliest valid start or `-1`.

8.6 Complexity

Sorting costs $O(n \log n)$, the scan is linear, and memory is $O(n)$.

8.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests; cin >> tests;
    while (tests--) {
        int n; long long duration, limit;
        cin >> n >> duration >> limit;
        vector<pair<long long, long long>> busy(n);
        for (auto &interval : busy) cin >> interval.first >> interval.second;
        sort(busy.begin(), busy.end());
        long long cur = 0;
        for (auto [left, right] : busy) {
            if (cur + duration - 1 < left) break;
            cur = max(cur, right + 1);
        }
        cout << (cur + duration - 1 <= limit ? cur : -1) << '\n';
    }
}
```

8.8 Full Python solution

```
import sys

data = list(map(int, sys.stdin.buffer.read().split()))
index = 1
out = []
for _ in range(data[0]):
    n, duration, limit = data[index:index + 3]
```

```

index += 3
busy = sorted((data[i], data[i + 1]) for i in range(index, index + 2 * n, 2))
index += 2 * n
cur = 0
for left, right in busy:
    if cur + duration - 1 < left:
        break
    cur = max(cur, right + 1)
out.append(str(cur if cur + duration - 1 <= limit else -1))
print("\n".join(out))

```

8.9 Tests that matter

Test no intervals, touching intervals such as [2,2] and [3,3], nested intervals, and a valid slot ending exactly at `limit`. The small oracle scans integer starts; the scale case checks the sorted scan is necessary.

9 E. Smallest Distinct Subsequence

9.1 Restatement and recognition

We must keep exactly one copy of every distinct character, preserve subsequence order, and make the resulting string lexicographically smallest. Choosing the first occurrence of every character is not sufficient: in `bcabc`, choosing the first `b` would force the answer to begin with `b`, although `abc` is possible.

The lexicographic objective suggests committing to the smallest safe next character. A monotonic stack implements that decision while allowing an earlier choice to be withdrawn only when another copy remains later.

9.2 Maintained information

While scanning left to right, maintain:

```
remaining[c] = occurrences of c not processed yet
used[c]      = whether c is currently in the answer stack
answer       = the chosen stack of distinct characters
```

After reading a character `c`, decrement `remaining[c]` first. If `c` is already used, skip it: adding it would violate the exactly-once condition. Otherwise, while the stack top is larger than `c` and occurs again later, pop it. The pop makes the answer lexicographically smaller and is safe precisely because `remaining[top] > 0`.

9.3 Algorithm

1. Count every character's occurrences.
2. Scan the string from left to right.
3. Decrease the current character's remaining count.
4. If it is already present in the stack, continue.
5. Pop while:
 - the top is lexicographically larger than the current character; and
 - another occurrence of the top remains.
6. Push the current character and mark it used.
7. Print the stack.

9.4 Correctness proof

First, every character in the stack is distinct because a used character is never pushed again. Every distinct input character is also present at the end: when its last occurrence is processed, it either is already used or is pushed; once no occurrence remains, the pop condition can never remove it.

It remains to prove lexicographic minimality. Consider processing a new character c . If a larger stack top x occurs later, replacing this occurrence of x by c now and taking x later preserves feasibility and makes the first changed answer position smaller. Therefore every performed pop is safe and necessary for a lexicographically minimal result. If popping stops, then the top is no larger than c , so moving c earlier would not improve the answer, or the top has no later occurrence, so removing it would make a valid answer impossible. Thus the algorithm makes the smallest feasible choice at every answer position, and the final subsequence is lexicographically smallest.

9.5 Complexity

Each input character is pushed at most once and popped at most once. The total time is $O(n)$. The stack uses $O(k)$ memory for $k \leq 26$ distinct lowercase characters; the two count arrays use $O(26)$.

9.6 Common wrong answers

- Popping a larger character without checking that it occurs later can lose a required distinct character.
- Forgetting `used` can produce duplicate characters.
- Decrementing `remaining[c]` after the pop loop incorrectly treats the current occurrence as still available in the future.
- A sorted set of distinct characters is not necessarily a subsequence.

9.7 Tests that matter

<code>aaaa</code>	<code>-> a</code>	(one distinct character)
<code>bcabc</code>	<code>-> abc</code>	(an early choice must be postponed)
<code>cbacdbc</code>	<code>-> acdb</code>	(not all larger characters can be popped)
<code>bbcaac</code>	<code>-> bac</code>	(last-occurrence condition matters)
<code>zyx</code>	<code>-> zyx</code>	(no character can be postponed)

The small randomized oracle can enumerate all subsequences, retain those with exactly the required character set, and choose the lexicographically smallest.

9.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--) {
        string s, answer;
        cin >> s;
        array<int, 26> remaining{}, used{};
        for (char c : s) ++remaining[c - 'a'];
        for (char c : s) {
            int x = c - 'a';
            --remaining[x];
            if (used[x]) continue;
            while (!answer.empty() && answer.back() > c &&
                    remaining[answer.back() - 'a'] > 0) {
                used[answer.back() - 'a'] = false;
                answer.pop_back();
            }
            answer.push_back(c);
            used[x] = true;
        }
        cout << answer << '\n';
    }
}
```

9.9 Full Python solution

```
import sys

def solve(s: str) -> str:
    remaining = [0] * 26
    for char in s:
        remaining[ord(char) - 97] += 1
    used = [False] * 26
    stack = []
    for char in s:
        index = ord(char) - 97
        remaining[index] -= 1
        if used[index]:
            continue
        while stack and stack[-1] > char and remaining[ord(stack[-1]) - 97]:
```

```
        used[ord(stack.pop()) - 97] = False
        stack.append(char)
        used[index] = True
    return "".join(stack)

def main() -> None:
    data = sys.stdin.read().split()
    print("\n".join(solve(s) for s in data[1:]))

if __name__ == "__main__":
    main()
```

10 F. Adjacent Cancellation

10.1 Recognition and stack state

Deleting a pair can create a new equal pair across the deletion boundary. A single left-to-right pass still works if the processed prefix is stored in its fully reduced form. Only its last surviving character can interact with the next input character, which is exactly a stack interface.

For each character c :

- if the stack top is c , pop it because the two adjacent copies cancel;
- otherwise push c .

The stack itself is the reduced prefix, so no second reduction pass is needed.

10.2 Correctness proof

We prove by induction that after processing any prefix, the stack equals that prefix after all possible cancellations. It is true for the empty prefix. When the next character c arrives, the old reduced prefix has no equal adjacent pair internally. The only possible new pair is between its last character and c . If they are equal, popping performs exactly that cancellation; otherwise appending c leaves the result reduced. Thus the invariant is preserved, and after the full string the stack is the requested final string.

Overlapping choices do not create ambiguity: the only overlap is three equal characters, and deleting either adjacent pair leaves the same one character; disjoint deletions commute.

10.3 Complexity, mistakes, and tests

Every character is pushed once and popped at most once, giving $O(n)$ time and $O(n)$ memory.

Do not repeatedly erase from the middle of a string: each erase may shift $O(n)$ characters. Remember to print `EMPTY` for an empty stack.

```
a      -> a
aa     -> EMPTY
aaa    -> a
abba   -> EMPTY      (a new pair appears after a deletion)
abab   -> abab
azxxzy -> ay
```

For random short strings, repeatedly scan for any equal adjacent pair until none remains and compare with the stack.

10.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--) {
        string s, stack;
        cin >> s;
        for (char c : s) {
            if (!stack.empty() && stack.back() == c) stack.pop_back();
            else stack.push_back(c);
        }
        cout << (stack.empty() ? "EMPTY" : stack) << '\n';
    }
}
```

10.5 Full Python solution

```
import sys

def main() -> None:
    data = sys.stdin.buffer.read().split()
    output = []
    for token in data[1:]:
        stack = []
        for char in token:
            if stack and stack[-1] == char:
                stack.pop()
            else:
                stack.append(char)
        output.append(bytes(stack).decode() if stack else "EMPTY")
    sys.stdout.write("\n".join(output))

if __name__ == "__main__":
    main()
```

11 G. Next Smaller Distance

11.1 From unresolved queries to a monotonic stack

At index i , we cannot answer until the first later value strictly below $a[i]$ appears. Keep indices whose answer is still unknown. When value $a[j]$ arrives, it resolves every suffix index with a larger value.

The unresolved stack has nondecreasing values from bottom to top. While $a[j] < a[\text{stack.top()}]$, pop index i and set its distance to $j-i$. Equal values stay: the problem asks for strictly smaller.

11.2 Correctness proof

When i is popped by j , $a[j] < a[i]$. No earlier position between them was smaller than $a[i]$; such a position would have popped i when it was processed. Therefore j is exactly the first smaller position for i .

If an index remains after the scan, no later processed value was smaller, so its correct output is zero. Each assigned and unassigned answer is therefore correct.

11.3 Complexity and common mistakes

Every index is pushed once and popped at most once: $O(n)$ time and $O(n)$ memory. The apparent nested `while` loop is linear in total, not quadratic.

- Pop on $<$, not $<=$.
- Store indices, because the output is a distance.
- Initialize unanswered positions to zero.
- A decreasing array is not quadratic: every new value performs one pop.

Increasing values $\rightarrow$ all zero.

Decreasing values $\rightarrow$ 1 for every position except the last.

All equal $\rightarrow$ all zero.

5 2 2 6 1 4 $\rightarrow$ duplicates and several pops at one index.

The oracle scans rightward independently from every position for small arrays.

11.4 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<long long> a(n);
    for (auto &value : a) cin >> value;
    vector<int> answer(n), stack;
    for (int i = 0; i < n; ++i) {
        while (!stack.empty() && a[i] < a[stack.back()]) {
            int index = stack.back();
            stack.pop_back();
            answer[index] = i - index;
        }
        stack.push_back(i);
    }
    for (int i = 0; i < n; ++i)
        cout << answer[i] << (i + 1 == n ? '\n' : ' ');
}

```

11.5 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, values = data[0], data[1:]
    answer = [0] * n
    stack = []
    for index, value in enumerate(values):
        while stack and value < values[stack[-1]]:
            previous = stack.pop()
            answer[previous] = index - previous
        stack.append(index)
    print(" ".join(map(str, answer)))

if __name__ == "__main__":
    main()

```

12 H. Smallest Fixed-Length Subsequence

12.1 Greedy interpretation

Choosing k characters is equivalent to deleting exactly $n-k$. While scanning, if the tentative answer ends with a character larger than the new character, deleting that larger character improves the first position where the answers differ. It is safe only while deletion capacity remains.

Maintain a stack and `removals = n-k`. Pop while:

```
removals > 0 and stack.top > current
```

Then push the current character. If removals remain after the scan, the stack is nondecreasing at every removable boundary, so delete the remaining suffix; equivalently print the first k stack characters.

12.2 Correctness proof

Whenever the algorithm pops $x > c$, replacing x by the later c uses one allowed deletion and produces a lexicographically smaller prefix, so no optimal answer that keeps x there is better. When the pop loop stops, either there is no deletion capacity or the previous character is at most c ; moving c earlier is then impossible or cannot improve the prefix.

Thus every deletion made during the scan is a safe earliest lexicographic improvement. If deletions remain, no inversion that can be improved remains, and deleting the latest characters preserves the smallest possible prefix. Exactly $n-k$ characters are removed, so the result has length k and is lexicographically minimal.

12.3 Complexity, pitfalls, and tests

Each character is pushed once and popped once at most: $O(n)$ time and $O(n)$ memory.

Unlike Problem E, duplicates are allowed and the target is a fixed length; there is no **used** set or remaining-occurrence condition. Do not forget the final suffix truncation when the input is already nondecreasing.

```
k = n          -> original string.
k = 1          -> smallest character.
Increasing s    -> take its prefix.
Decreasing s    -> repeated pops.
Repeated chars -> strict comparison preserves equal choices.
```

For $n \leq 14$, enumerate every combination of k indices and take the minimum.

12.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s, answer;
    int k;
    cin >> s >> k;
    int removals = (int)s.size() - k;
    for (char c : s) {
        while (removals && !answer.empty() && answer.back() > c) {
            answer.pop_back();
            --removals;
        }
        answer.push_back(c);
    }
    answer.resize(k);
    cout << answer << '\n';
}
```

12.5 Full Python solution

```
import sys

def main() -> None:
    data = sys.stdin.buffer.read().split()
    s, k = data[0].decode(), int(data[1])
    removals = len(s) - k
    answer = []
    for char in s:
        while removals and answer and answer[-1] > char:
            answer.pop()
            removals -= 1
        answer.append(char)
    print("".join(answer[:k]))

if __name__ == "__main__":
    main()
```

13 I. Wildcard Parentheses

13.1 How to discover the compression

It is not natural to guess `low` and `high` immediately. Begin with the straightforward balance DP. After each prefix, store every balance reachable without having used a negative prefix:

```
possible = {0}

for each character:
    create next_possible
    for every balance in possible:
        '(' adds balance + 1
        ')' adds balance - 1 when it is nonnegative
        '?' tries both choices
    possible = next_possible
```

This model is correct, but there can be $O(n)$ balances after each of n characters, so an explicit implementation may take $O(n^2)$.

Before optimizing, write several DP rows:

prefix	possible balances
empty	{0}
?	{1}
??	{0, 2}
???	{1, 3}
????	{0, 2, 4}

Fixed characters merely shift a complete row:

before '('	: {0, 2, 4}
after '('	: {1, 3, 5}
before ')'	: {1, 3, 5}
after ')'	: {0, 2, 4}

The rows do not appear to be arbitrary sets such as {0, 2, 8}. They contain every balance of the correct parity between their smallest and largest values. This observation suggests compressing one whole DP row to its two endpoints:

```
low  = smallest reachable balance
high = largest reachable balance
```

The natural development is therefore:

```
complete  $O(n^2)$  balance DP
    -> inspect several rows
    -> conjecture a step-two interval
    -> store only its endpoints
    -> prove that every transition preserves the shape
```

The induction below is the verification of that observed structure, not something that must be guessed before constructing the slow DP.

13.2 Feasible-balance interval

For a prefix, let balance be opens minus closes. Validity requires every prefix balance to be nonnegative and the final balance to be zero. A wildcard changes the balance by either -1 or $+1$.

Track the smallest and largest reachable nonnegative balances, `low` and `high`. All reachable balances have the parity of the prefix length:

```
'(' : low++, high++
')' : low--, high--
'?' : low--, high++
```

If `high` < 0 , even the most opening-heavy assignment fails. If `low` < 0 , replace it by the smallest nonnegative balance of the current prefix parity: zero for an even prefix, one for an odd prefix. The full length must be even, and the answer is possible exactly when final `low` $== 0$.

13.3 Correctness proof

Let $R[i]$ be the balances reachable after the first i characters using only replacements whose prefixes have never had negative balance. We prove by induction that $R[i]$ is empty or has the form:

```
{low, low + 2, low + 4, ..., high}
```

Before reading anything, $R[0] = \{0\}$, so the property holds.

Assume it holds before the next character. A fixed `(` adds one to every balance, preserving a step-two sequence. A fixed `)` subtracts one. If this creates -1 , removing that first invalid value still leaves a step-two sequence.

For a wildcard, write the old set as:

```
{low, low + 2, ..., low + 2*t}
```

Choosing `)` produces:

```
down = {low - 1 + 2*r : r = 0, 1, ..., t}
```

Choosing (produces:

```
up = {low - 1 + 2*r : r = 1, 2, ..., t + 1}
```

Thus `down` covers the index range $0..t$, while `up` covers $1..t+1$. Their union covers every index $0..t+1$, so it contains every step-two value from `low-1` through `low+2*t+1`.

When $t \geq 1$, the two shifted sets overlap on indices $1..t$. When $t = 0$, they do not overlap, but their only values are `low-1` and `low+1`, which differ by exactly two and are therefore consecutive values of the required parity. In neither case is there a gap.

If `low = 0`, the first down-shifted value is `-1`. Discarding it merely removes the first element of the step-two sequence; it cannot create an internal gap. If even `high < 0`, no nonnegative balance is reachable. Therefore the induction property is preserved for every possible character.

At the end, balance zero is reachable exactly when the length is even and the minimum reachable balance is zero. Such an assignment has no negative prefix by construction, so it is a valid parenthesis sequence.

13.4 Complexity, mistakes, and tests

Time is $O(n)$ and memory is $O(1)$ per string.

Checking only the final number of opens misses invalid prefixes. Treating ? as if it may be empty solves a different problem; the even-length and parity conditions matter.

```
Odd length -> always NO.
??          -> YES.
)( and ?)( -> invalid prefix.
((??))      -> several reachable balances.
All '?' with even and odd lengths.
```

The small oracle enumerates all $2^{(\text{number of ?})}$ replacements.

13.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--) {
        string s;
```

```

cin >> s;
int low = 0, high = 0;
bool possible = s.size() % 2 == 0;
for (int index = 0; index < (int)s.size(); ++index) {
    char c = s[index];
    if (c == '(') ++low, ++high;
    else if (c == ')') --low, --high;
    else --low, ++high;
    if (low < 0) low = (index + 1) & 1;
    if (high < 0) possible = false;
}
cout << (possible && low == 0 ? "YES" : "NO") << '\n';
}
}

```

13.6 Full Python solution

```

import sys

def possible(s):
    if len(s) % 2:
        return False
    low = high = 0
    for index, char in enumerate(s):
        if char == 40:
            low += 1
            high += 1
        elif char == 41:
            low -= 1
            high -= 1
        else:
            low -= 1
            high += 1
        if low < 0:
            low = (index + 1) & 1
        if high < 0:
            return False
    return low == 0

def main() -> None:
    data = sys.stdin.buffer.read().split()
    print("\n".join("YES" if possible(s) else "NO" for s in data[1:]))

if __name__ == "__main__":
    main()

```

14 J. Interval Point Cover

14.1 Greedy choice

Sort intervals by increasing right endpoint. For the first interval not covered by the most recently selected point, choose that interval's right endpoint. This point lies as far right as possible while still covering the urgent interval, maximizing its chance to cover later intervals.

Only the last selected point is needed. If it is smaller than the next interval's left endpoint, that interval is uncovered and forces a new choice.

14.2 Exchange proof

Consider the first interval $[l, r]$ not already covered. Every valid solution must choose some point p inside it. Replace the first such point by r . Because intervals are processed by right endpoint, every remaining interval has right endpoint at least r . Any remaining interval that contained p and starts no later than r also contains r ; intervals starting after r were not covered by p anyway. The replacement covers no fewer relevant intervals.

Therefore an optimal solution exists containing the greedy point r . Remove all intervals it covers and repeat the same argument, proving the greedy count optimal.

14.3 Complexity and tests

Sorting costs $O(n \log n)$; the scan is $O(n)$, and memory is $O(n)$.

Closed endpoints mean `last == left` is covered. Sorting by left endpoint or choosing the left endpoint does not support the exchange argument.

```
One interval.  
All intervals share one endpoint -> 1.  
Pairwise disjoint intervals -> n.  
Nested intervals.  
Touching closed intervals.  
Negative coordinates and duplicate intervals.
```

For small coordinate ranges, enumerate subsets of candidate points and find the minimum hitting set.

14.4 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<pair<long long, long long>> intervals(n);
    for (auto &[left, right] : intervals) cin >> left >> right;
    sort(intervals.begin(), intervals.end(),
        [](auto a, auto b) { return tie(a.second, a.first) < tie(b.second, b.first); });
    long long last = LLONG_MIN;
    int answer = 0;
    for (auto [left, right] : intervals) {
        if (last < left) {
            last = right;
            ++answer;
        }
    }
    cout << answer << '\n';
}

```

14.5 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    intervals = list(zip(data[1::2], data[2::2]))
    intervals.sort(key=lambda interval: (interval[1], interval[0]))
    last = None
    answer = 0
    for left, right in intervals:
        if last is None or last < left:
            last = right
            answer += 1
    print(answer)

if __name__ == "__main__":
    main()

```

15 K. Deadline Selection

15.1 Greedy subset maintenance

Any chosen set can be scheduled in nondecreasing deadline order: swapping an inverted adjacent pair cannot make either completion deadline harder. Process jobs in that order.

Tentatively select every job and maintain total duration in a maximum heap. If the current deadline is exceeded, remove the selected job with the largest duration. Removing one job suffices: before adding the new job, the selected set was feasible, and the removed maximum is at least as long as the new job.

15.2 Correctness proof

After each processed deadline, maintain this invariant: the heap contains a maximum-cardinality feasible subset of processed jobs, and among such subsets its total duration is minimum.

Adding a job increases the candidate cardinality by one. If it remains feasible, no processed subset can have larger cardinality. If it is infeasible, no subset containing all current selected jobs is feasible, so at most the previous cardinality can be retained. Removing the longest selected duration restores feasibility and gives the minimum possible total among subsets formed by deleting one job. This preserves both parts of the invariant. By induction, the final heap size is the maximum feasible job count.

15.3 Complexity, pitfalls, and tests

Sorting and heap operations cost $O(n \log n)$ time; memory is $O(n)$. The total duration requires 64-bit storage.

This objective maximizes the number of jobs, not profit or minimizes lateness. Removing the most recently added job is not always best; remove the longest selected duration.

```
Every job feasible.  
No job feasible alone.  
One long early job replaced by several short jobs.  
Equal deadlines.  
Deadlines already unsorted in input.  
Large durations -> 64-bit total.
```

For small n , enumerate every subset and check it in deadline order.

15.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<pair<long long, long long>> jobs(n);
    for (auto &[duration, deadline] : jobs) cin >> duration >> deadline;
    sort(jobs.begin(), jobs.end(),
        [](auto a, auto b) { return tie(a.second, a.first) < tie(b.second, b.first); });
    priority_queue<long long> selected;
    long long total = 0;
    for (auto [duration, deadline] : jobs) {
        selected.push(duration);
        total += duration;
        if (total > deadline) {
            total -= selected.top();
            selected.pop();
        }
    }
    cout << selected.size() << '\n';
}
```

15.5 Full Python solution

```
import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    jobs = list(zip(data[1::2], data[2::2]))
    jobs.sort(key=lambda job: (job[1], job[0]))
    selected = []
    total = 0
    for duration, deadline in jobs:
        heapq.heappush(selected, -duration)
        total += duration
        if total > deadline:
            total += heapq.heappop(selected)
    print(len(selected))
```

```
if __name__ == "__main__":  
    main()
```

16 L. Smallest After Deletions

16.1 From H to an explicit deletion budget

H asks for a subsequence of length k . Here the statement instead says to delete k characters. These are the same structural choice: after the deletions, the answer has a fixed length.

The useful question while scanning a new digit c is:

```
If the last chosen digit is larger than  $c$ , can I spend one deletion
to replace that larger digit by  $c$ ?
```

If a deletion remains, the answer is yes. Removing the larger last digit makes the first changed position smaller, so every possible continuation becomes lexicographically better.

Maintain the chosen prefix as a stack. While its last digit is larger than the current digit and deletions remain, pop. Then append the current digit.

16.2 Why unused deletions come from the end

For 11234, no later digit is smaller, so the popping rule spends nothing. We still must delete characters. Among two adjacent retained digits that stay in increasing order, deleting the later one preserves the earlier, more important prefix. Therefore all remaining deletions should remove a suffix of the stack.

For example:

```
s = 1432219, deletions = 3

read 1: 1
read 4: 14
read 3: pop 4, then 13
read 2: pop 3, then 12
read 2: 122
read 1: pop 2, then 121
read 9: 1219
```

16.3 Correctness proof

Whenever the stack ends in a and the current digit is $b < a$, keeping a while deleting b is worse than deleting a and keeping b : the two results have the same prefix before that position, and the replacement has the smaller next digit. The pop is legal because it spends one available deletion.

After no such inversion remains, the current digit cannot improve an earlier position, so appending it is safe. If deletions remain after the scan, the stack is nondecreasing wherever a deletion could still have helped. Deleting from its end leaves the longest possible prefix unchanged, and is therefore optimal. Thus the final string is the smallest obtainable one.

16.4 Complexity, pitfalls, and tests

Every digit is pushed once and popped at most once: $O(n)$ time and $O(n)$ memory.

Leading zeroes are part of the answer; do not convert the string to an integer. Equal digits should not trigger a pop. Remember the final suffix deletion.

```
Already increasing.  
Strictly decreasing.  
All digits equal.  
A zero after a nonzero prefix.  
k = 0 and k = n - 1.
```

16.5 Full C++ solution

```
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
    ios::sync_with_stdio(false);  
    cin.tie(nullptr);  
    string s, answer;  
    int k;  
    cin >> s >> k;  
    for (char c : s) {  
        while (k > 0 && !answer.empty() && answer.back() > c) {  
            answer.pop_back();  
            --k;  
        }  
        answer.push_back(c);  
    }  
    answer.resize(answer.size() - k);  
    cout << answer << '\n';  
}
```

16.6 Full Python solution

```
import sys
```

```
def main() -> None:
    s, raw_k = sys.stdin.buffer.read().split()
    k = int(raw_k)
    answer = []
    for digit in s.decode():
        while k and answer and answer[-1] > digit:
            answer.pop()
            k -= 1
        answer.append(digit)
    if k:
        del answer[-k:]
    print("".join(answer))

if __name__ == "__main__":
    main()
```

17 M. Required Letter Subsequence

17.1 Why H's single feasibility check is no longer enough

Without the letter requirement, a pop is allowed whenever the remaining input can still fill k positions. Now a choice can fail in two independent ways:

```
not enough characters remain to reach length  $k$ ;  
not enough required letters remain to reach  $r$  copies.
```

This is the intended extra difficulty. We retain the monotonic-stack idea, but every pop and every push must preserve both resources.

Maintain:

```
answer          chosen stack  
chosen_required  required letters currently in answer  
remaining_required required letters at the current position or later
```

At position i , popping the last character leaves $\text{len}(\text{answer}) - 1$ chosen characters and $n - i$ input characters still usable, including the current one. The pop can still reach length k exactly when their sum is at least k .

If the popped character is required, also demand that the required copies already chosen after the pop plus all required copies still available is at least r .

17.2 Deciding whether to push

If the current character is required and the answer is not full, keep it. Extra copies are harmless and might also be lexicographically small.

For a non-required character, reserve enough empty positions for the missing required copies:

```
positions left after this push  $\geq r - \text{chosen\_required}$ 
```

This reservation is what prevents an attractive early letter from making the final constraint impossible.

17.3 Correctness proof

Consider a pop performed before current character c . The popped character is larger than c , so replacing it makes the first differing answer position smaller. The length check guarantees the remaining input can fill all answer positions. The required-count check guarantees the remaining input can still supply enough required letters. Hence an optimal completion still exists after every pop.

When a non-required character is skipped, accepting it would consume a slot needed for the still-missing required copies. No valid completion can use that character in that slot. When a character is accepted, all constraints remain feasible by the same reservation checks.

Therefore the algorithm takes every safe lexicographic improvement and rejects only choices that cannot belong to a valid completion. The final length- k subsequence is valid and lexicographically smallest.

17.4 Complexity, pitfalls, and tests

The scan is $O(n)$ because each character is pushed and popped at most once. Memory is $O(k)$.

`remaining_required` must still include the current character during the pop loop; decrement it only after processing that position. A pop of the required letter must update `chosen_required`. Test cases where all reserved slots are forced are especially important.

```
r = k, so every chosen character is required.  
Exactly r required copies exist.  
The required letter is lexicographically large.  
Several required letters are popped and later replaced.  
k = n.
```

17.5 Full C++ solution

```
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
    ios::sync_with_stdio(false);  
    cin.tie(nullptr);  
    string s, answer;  
    int k, r;  
    char required;  
    cin >> s >> k >> required >> r;  
    int remaining_required = count(s.begin(), s.end(), required);  
    int chosen_required = 0;  
    for (int i = 0; i < (int)s.size(); ++i) {  
        char c = s[i];  
        while (!answer.empty() && answer.back() > c  
            && (int)answer.size() - 1 + (int)s.size() - i >= k  
            && (answer.back() != required
```

```

        || chosen_required - 1 + remaining_required >= r)) {
    if (answer.back() == required) --chosen_required;
    answer.pop_back();
}
if ((int)answer.size() < k) {
    if (c == required) {
        answer.push_back(c);
        ++chosen_required;
    } else if (k - (int)answer.size() - 1 >= r - chosen_required) {
        answer.push_back(c);
    }
}
if (c == required) --remaining_required;
}
cout << answer << '\n';
}

```

17.6 Full Python solution

```

import sys

def main() -> None:
    data = sys.stdin.buffer.read().split()
    s = data[0].decode()
    k = int(data[1])
    required = data[2].decode()
    r = int(data[3])
    remaining_required = s.count(required)
    chosen_required = 0
    answer = []
    for i, char in enumerate(s):
        while (answer and answer[-1] > char
               and len(answer) - 1 + len(s) - i >= k
               and (answer[-1] != required
                   or chosen_required - 1 + remaining_required >= r)):
            if answer[-1] == required:
                chosen_required -= 1
            answer.pop()
        if len(answer) < k:
            if char == required:
                answer.append(char)
                chosen_required += 1
            elif k - len(answer) - 1 >= r - chosen_required:
                answer.append(char)
            if char == required:
                remaining_required -= 1
    print("".join(answer))

```

```
if __name__ == "__main__":  
    main()
```

18 N. Wildcard Choice Map

18.1 Ask about one position without fixing all the others

I answers whether some valid completion exists. Here, for every question mark, we must answer two separate questions:

```
Can this position be ( in some valid completion?  
Can this position be ) in some valid completion?
```

Constructing one completion cannot answer this: a position used as (in one solution might also be usable as) in another. Instead, compute feasibility information on both sides of every position.

For every prefix ending before position *i*, store the smallest and largest nonnegative reachable balances:

```
prefix_low[i], prefix_high[i]
```

As proved in I, all balances of the correct parity between these bounds are reachable. Compute these arrays with the same forward transitions and lower bound of zero.

18.2 Backward balances

For every suffix starting at *i*, store the balances that may exist immediately before that suffix and can still finish at balance zero without ever becoming negative:

```
suffix_low[i], suffix_high[i]
```

Start after the string with the singleton balance zero and reverse each transition:

```
current ( : predecessor = next balance - 1, and it must be nonnegative  
current ) : predecessor = next balance + 1  
current ? : union of both possibilities
```

These predecessor sets are again step-two intervals. For ?, consecutive next balances create overlapping predecessor pairs, exactly as in I's forward proof. An empty interval must stay empty; later characters cannot repair a prefix or suffix for which no balance exists.

18.3 Test each replacement by intersection

Suppose position i is $?$.

For an opening choice, every reachable balance before it increases by one. Intersect that shifted prefix interval with the suffix interval after i . The choice is possible exactly when the intersection is nonempty.

For a closing choice, first discard prefix balance zero because closing there would go negative. Shift every remaining prefix balance down by one and perform the same intersection.

All compared balances have the same forced parity, so ordinary interval intersection is sufficient:

```
max(left endpoints) <= min(right endpoints)
```

If only one choice intersects, print that parenthesis. If both intersect, print $?$. If balance zero is not reachable after the full string, print **IMPOSSIBLE**.

18.4 Correctness proof

The forward interval contains exactly the balances attainable by some valid replacement of the prefix. The backward interval contains exactly the balances from which some replacement of the suffix reaches zero without a negative balance. These claims follow by induction from the forward and reversed transitions, with the same overlap argument used in I.

If the shifted prefix interval and suffix interval intersect at balance b , choose a prefix assignment reaching the corresponding balance before the tested character, make the tested replacement, then choose a suffix assignment that finishes from b . Their concatenation is a valid completion containing that choice.

Conversely, any valid completion containing that choice reaches some balance b after it. That b must belong to both the shifted prefix set and the suffix set, so the intervals intersect. Thus the two tests classify every question mark exactly.

18.5 Complexity, pitfalls, and tests

Both directional scans and all intersections take $O(n)$ time. The four bound arrays use $O(n)$ memory.

Do not infer flexibility from one constructed completion. For a closing choice, exclude starting balance zero. Preserve empty intervals explicitly: using their numerical bounds in another transition can falsely make them nonempty.

```
No valid completion.  
All question marks.  
A position forced open at the beginning.  
A position forced closed at the end.  
Both choices possible in the middle.  
No question marks.
```

18.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

bool intersects(int low1, int high1, int low2, int high2) {
    return max(low1, low2) <= min(high1, high2);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    int n = s.size();
    vector<int> prefix_low(n + 1), prefix_high(n + 1);
    for (int i = 0; i < n; ++i) {
        if (prefix_low[i] > prefix_high[i]) {
            prefix_low[i + 1] = 1;
            prefix_high[i + 1] = 0;
            continue;
        }
        int low = prefix_low[i];
        int high = prefix_high[i];
        if (s[i] == '(') ++low, ++high;
        else if (s[i] == ')') --low, --high;
        else --low, ++high;
        int parity = (i + 1) & 1;
        if (low < 0) low = parity;
        prefix_low[i + 1] = low;
        prefix_high[i + 1] = high;
    }
    if (n % 2 || prefix_low[n] > 0 || prefix_high[n] < 0) {
        cout << "IMPOSSIBLE\n";
        return 0;
    }

    vector<int> suffix_low(n + 1), suffix_high(n + 1);
    for (int i = n - 1; i >= 0; --i) {
        if (suffix_low[i + 1] > suffix_high[i + 1]) {
            suffix_low[i] = 1;
            suffix_high[i] = 0;
            continue;
        }
        int low = suffix_low[i + 1];
        int high = suffix_high[i + 1];
        if (s[i] == '(') {
            low = low == 0 ? 1 : low - 1;
            high -= 1;
        } else if (s[i] == ')') {

```

```

        ++low;
        ++high;
    } else {
        low = low == 0 ? 1 : low - 1;
        ++high;
    }
    suffix_low[i] = low;
    suffix_high[i] = high;
}

string answer = s;
for (int i = 0; i < n; ++i) {
    if (s[i] != '?') continue;
    bool can_open = intersects(
        prefix_low[i] + 1, prefix_high[i] + 1,
        suffix_low[i + 1], suffix_high[i + 1]);
    int closing_low = max(prefix_low[i], 1) - 1;
    bool can_close = intersects(
        closing_low, prefix_high[i] - 1,
        suffix_low[i + 1], suffix_high[i + 1]);
    if (can_open && !can_close) answer[i] = '(';
    else if (!can_open && can_close) answer[i] = ')';
    else answer[i] = '?';
}
cout << answer << '\n';
}

```

18.7 Full Python solution

```

import sys

def main() -> None:
    s = sys.stdin.buffer.readline().strip().decode()
    n = len(s)
    prefix_low = [0] * (n + 1)
    prefix_high = [0] * (n + 1)
    for i, char in enumerate(s):
        if prefix_low[i] > prefix_high[i]:
            prefix_low[i + 1], prefix_high[i + 1] = 1, 0
            continue
        low, high = prefix_low[i], prefix_high[i]
        if char == "(":
            low += 1
            high += 1
        elif char == ")":
            low -= 1
            high -= 1
        else:

```

```

        low -= 1
        high += 1
    if low < 0:
        low = (i + 1) & 1
    prefix_low[i + 1] = low
    prefix_high[i + 1] = high

if n % 2 or not prefix_low[n] <= 0 <= prefix_high[n]:
    print("IMPOSSIBLE")
    return

suffix_low = [0] * (n + 1)
suffix_high = [0] * (n + 1)
for i in range(n - 1, -1, -1):
    if suffix_low[i + 1] > suffix_high[i + 1]:
        suffix_low[i], suffix_high[i] = 1, 0
        continue
    low, high = suffix_low[i + 1], suffix_high[i + 1]
    if s[i] == "(":
        low = 1 if low == 0 else low - 1
        high -= 1
    elif s[i] == ")":
        low += 1
        high += 1
    else:
        low = 1 if low == 0 else low - 1
        high += 1
    suffix_low[i], suffix_high[i] = low, high

answer = list(s)
for i, char in enumerate(s):
    if char != "?":
        continue
    after_low, after_high = suffix_low[i + 1], suffix_high[i + 1]
    can_open = max(prefix_low[i] + 1, after_low) <= min(
        prefix_high[i] + 1, after_high)
    closing_low = max(prefix_low[i], 1) - 1
    can_close = max(closing_low, after_low) <= min(
        prefix_high[i] - 1, after_high)
    if can_open and not can_close:
        answer[i] = "("
    elif can_close and not can_open:
        answer[i] = ")"
print("".join(answer))

if __name__ == "__main__":
    main()

```

19 O. Minimum Parenthesis Depth

19.1 Turn optimization into a feasibility question

For a proposed depth limit D , ask:

```
Can every ? be replaced so that all prefix balances stay between 0 and D
and the final balance is 0?
```

If limit D is feasible, every larger limit is also feasible. This monotonic property permits binary search for the smallest feasible limit.

The feasibility check extends I 's reachable-balance interval. After each character, reachable balances have one parity and no gaps when viewed in steps of two. Store their smallest and largest values as `low` and `high`.

Transitions are:

```
( : low += 1, high += 1
) : low -= 1, high -= 1
? : low -= 1, high += 1
```

Then intersect the reachable set with balances from 0 through D . The balance after a prefix of length i must have the same odd/even parity as i , so round each boundary inward to that parity. If `low` > `high`, this limit is impossible.

19.2 Why the compressed interval remains exact

Suppose the previous reachable balances are

```
low, low + 2, ..., high.
```

For (or), shifting every value by one clearly preserves this form. For ?, each old balance b produces $b - 1$ and $b + 1$. Consecutive old balances differ by two:

```
from b:      b - 1, b + 1
from b + 2:  b + 1, b + 3
```

The two produced sets overlap at $b + 1$, so no gap appears. Intersecting with the lower and upper depth bounds only removes a prefix or suffix of this step-two list. Therefore `low` and `high`, together with the forced parity, describe all and only reachable balances.

At the end, zero is reachable exactly when `low` is zero. Binary search then returns the smallest depth whose feasibility check succeeds.

19.3 Worked example

For $s = \text{????}$ and depth limit 1:

```
prefix 1: reachable within limit = {1}
prefix 2: reachable within limit = {0}
prefix 3: reachable within limit = {1}
prefix 4: reachable within limit = {0}
```

So depth one is feasible using $()()$. For $s = ()()$, limit one fails after the second character, while limit two succeeds.

19.4 Complexity, pitfalls, and tests

Each feasibility check is $O(n)$, and binary search performs $O(\log n)$ checks, for $O(n \log n)$ time and $O(1)$ auxiliary memory.

Clipping high to D without restoring the required parity can invent an unreachable balance. Odd-length strings are impossible. First verify that any completion exists using a generous limit before binary searching.

```
All question marks.
Forced deeply nested prefix.
Impossible fixed prefix.
Odd length.
Minimum depth equal to 1 and equal to  $n / 2$ .
```

19.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

bool feasible(const string &s, int limit) {
    int low = 0, high = 0;
    for (int i = 0; i < (int)s.size(); ++i) {
        if (s[i] == '(') ++low, ++high;
        else if (s[i] == ')') --low, --high;
        else --low, ++high;
        int parity = (i + 1) & 1;
        if (low < 0) low = parity;
        high = min(high, limit);
        if ((high & 1) != parity) --high;
        if (low > high) return false;
    }
    return low == 0;
}
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string s;
    cin >> s;
    if (s.size() % 2 || !feasible(s, (int)s.size())) {
        cout << -1 << '\n';
        return 0;
    }
    int low = 1, high = s.size();
    while (low < high) {
        int middle = low + (high - low) / 2;
        if (feasible(s, middle)) high = middle;
        else low = middle + 1;
    }
    cout << low << '\n';
}

```

19.6 Full Python solution

```

import sys

def feasible(s: str, limit: int) -> bool:
    low = high = 0
    for prefix, char in enumerate(s, 1):
        if char == "(":
            low += 1
            high += 1
        elif char == ")":
            low -= 1
            high -= 1
        else:
            low -= 1
            high += 1
        parity = prefix & 1
        if low < 0:
            low = parity
        high = min(high, limit)
        if high & 1 != parity:
            high -= 1
        if low > high:
            return False
    return low == 0

def main() -> None:
    s = sys.stdin.buffer.readline().strip().decode()

```

```
if len(s) % 2 or not feasible(s, len(s)):
    print(-1)
    return
low, high = 1, len(s)
while low < high:
    middle = (low + high) // 2
    if feasible(s, middle):
        high = middle
    else:
        low = middle + 1
print(low)

if __name__ == "__main__":
    main()
```

20 P. Common Deadline

20.1 Isolate the exchange behind deadline selection

Every chosen task has the same deadline, so only the sum of chosen durations matters. If we want to fit exactly c tasks, the smallest possible total duration is obtained by choosing the c shortest tasks.

This suggests sorting all durations and taking the longest prefix whose sum does not exceed T .

For:

```
durations = 6, 2, 4, 3, 8  
T = 10  
sorted = 2, 3, 4, 6, 8
```

The first three total 9; a fourth would total at least 15. The answer is three.

20.2 Exchange proof

Take any feasible set of c tasks. If it contains duration a while an unchosen shorter duration $b < a$ exists, exchange a for b . The number of tasks stays c and total duration does not increase, so feasibility is preserved.

Repeating this exchange transforms some feasible set of size c into the c globally shortest tasks. Therefore, if the sum of the c shortest tasks exceeds T , no set of c tasks can fit. Conversely, if that sum is at most T , those tasks themselves are a feasible set.

The largest feasible sorted prefix is therefore exactly the optimum.

20.3 Connection to K

P has one global capacity constraint. K reveals deadline constraints one by one. In P, sorting once tells us which durations to retain. In K, a maximum heap maintains that same “retain the shortest durations” choice dynamically inside each deadline prefix.

20.4 Complexity, pitfalls, and tests

Sorting costs $O(n \log n)$ time and $O(n)$ memory. The sum needs 64-bit storage because it can exceed 32-bit range.

Do not sort by input order or greedily accept a long task before inspecting shorter ones. Because durations are positive and sorted, once the next task does not fit, no later task can fit either.

```
No task fits.  
Every task fits.  
Many equal durations.  
One very long task appears first.  
T near  $10^{18}$ .
```

20.5 Full C++ solution

```
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
    ios::sync_with_stdio(false);  
    cin.tie(nullptr);  
    int n;  
    long long limit;  
    cin >> n >> limit;  
    vector<long long> durations(n);  
    for (long long &duration : durations) cin >> duration;  
    sort(durations.begin(), durations.end());  
    long long total = 0;  
    int answer = 0;  
    for (long long duration : durations) {  
        if (total + duration > limit) break;  
        total += duration;  
        ++answer;  
    }  
    cout << answer << '\n';  
}
```

20.6 Full Python solution

```
import sys  
  
def main() -> None:  
    data = list(map(int, sys.stdin.buffer.read().split()))  
    limit = data[1]  
    total = answer = 0
```

```
for duration in sorted(data[2:]):
    if total + duration > limit:
        break
    total += duration
    answer += 1
print(answer)

if __name__ == "__main__":
    main()
```

21 Q. Event Attendance

21.1 Two orders matter

An event cannot be considered before its start day, but among events currently available, the one with the earliest end day is most urgent. This produces two data structures:

```
events sorted by start day;  
a minimum heap of end days for events that have started.
```

On each used day:

1. Add every event whose start is at most today.
2. Remove heap entries whose end is before today; they have expired.
3. Attend the available event with the smallest end day.
4. Advance to the next day.

When the heap is empty, jumping directly to the next start day avoids scanning up to 10^9 empty days.

21.2 Exchange proof

Suppose an optimal schedule attends event A today while another available event B ends no later than A. If B is never attended, replace A by B today; the number attended is unchanged.

If B is attended on a later day, swap their attendance days. B can be attended today because it is already available. Event A can be attended on B's old day: A had already started today, and its end is at least B's end, which includes that later day. The swap preserves feasibility and the number of events.

Thus some optimal schedule makes the greedy earliest-ending choice today. Applying the same argument after every chosen day proves the algorithm optimal.

21.3 Worked example

```
events: [1,3], [1,1], [2,2], [10,10]  
  
day 1: [1,1] is most urgent  
day 2: add [2,2], then attend it  
day 3: attend [1,3]  
heap empty: jump to day 10 and attend [10,10]  
answer = 4
```

Choosing the longest interval first would lose the event ending on day one. Sorting only by end day also fails operationally because future events are not yet available.

21.4 Complexity, pitfalls, and tests

Sorting costs $O(n \log n)$. Every event enters and leaves the heap once, so heap work is also $O(n \log n)$. Memory is $O(n)$.

Intervals are inclusive: an event with `end == day` is still attendable, so expire only `end < day`. Add newly starting events before choosing for that day. Use a day type that safely holds the input range and one increment.

```
Several events available on one day.  
Long flexible event versus short urgent events.  
Large gaps between start days.  
All events have the same interval.  
Single-day intervals and touching boundaries.
```

The random oracle treats events and days as a small bipartite matching instance, independently checking the greedy result.

21.5 Full C++ solution

```
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
    ios::sync_with_stdio(false);  
    cin.tie(nullptr);  
    int n;  
    cin >> n;  
    vector<pair<int, int>> events(n);  
    for (auto &[start, finish] : events) cin >> start >> finish;  
    sort(events.begin(), events.end());  
    priority_queue<int, vector<int>, greater<int>> finishes;  
    int index = 0, answer = 0;  
    long long day = 0;  
    while (index < n || !finishes.empty()) {  
        if (finishes.empty()) day = max(day, (long long)events[index].first);  
        while (index < n && events[index].first <= day) {  
            finishes.push(events[index].second);  
            ++index;  
        }  
        while (!finishes.empty() && finishes.top() < day) finishes.pop();  
        if (!finishes.empty()) {  
            finishes.pop();  
            ++answer;  
            ++day;  
        }  
    }  
}
```

```

    }
}
cout << answer << '\n';
}

```

21.6 Full Python solution

```

import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    events = sorted(zip(data[1::2], data[2::2]))
    finishes = []
    index = answer = 0
    day = 0
    while index < len(events) or finishes:
        if not finishes:
            day = max(day, events[index][0])
            while index < len(events) and events[index][0] <= day:
                heapq.heappush(finishes, events[index][1])
                index += 1
            while finishes and finishes[0] < day:
                heapq.heappop(finishes)
        if finishes:
            heapq.heappop(finishes)
            answer += 1
            day += 1
    print(answer)

if __name__ == "__main__":
    main()

```

22 R. Coverage Patches

22.1 Discover the state before the choice

Suppose the processed values can form every sum from 1 through `covered`. The next sum we must make is therefore exactly `covered + 1`.

If the next sorted input value `x` satisfies `x <= covered + 1`, adding it to any old subset sum extends continuous coverage through `covered + x`:

```
old sums:          0 through covered
old sums plus x:    x through x + covered
```

Because `x <= covered + 1`, these ranges touch or overlap. There is no gap.

If `x > covered + 1`, no later input value can form `covered + 1` either. Some patch is forced now.

22.2 Why the forced patch is `covered + 1`

A patch larger than `covered + 1` leaves that first missing sum uncovered. A smaller patch `p` is legal, but extends the covered range only to `covered + p`, which is no farther than using `covered + 1`.

Thus `covered + 1` is both necessary enough to fill the gap and maximal among patches that can fill it. After adding it, coverage becomes:

```
0 through 2 * covered + 1
```

This gives the complete greedy algorithm: sort the input; consume a value when it touches the covered range, otherwise patch the first gap.

For the sample:

```
values = 1, 5, 10; target = 20

use 1    -> covered = 1
patch 2  -> covered = 3
patch 4  -> covered = 7
use 5    -> covered = 12
use 10   -> covered = 22
```

Two patches are required.

22.3 Correctness proof

Maintain the invariant that every sum from zero through `covered` is representable using processed values and patches.

When the next value is at most `covered + 1`, the two ranges described above overlap, so consuming it preserves the invariant and extends its endpoint.

Otherwise `covered + 1` cannot be formed from processed values, and every unprocessed value is too large. Every solution must add a patch no larger than `covered + 1`. Replacing that solution's first such patch by exactly `covered + 1` covers at least as far and uses the same number of patches. Therefore an optimal solution exists making the greedy patch. Repeating this argument proves the final patch count minimum.

22.4 Complexity, pitfalls, and tests

Sorting costs $O(n \log n)$ time; the scan is $O(n)$, and memory is $O(n)$. Use 64-bit integers for values, the target, and `covered`.

The collection represents sums by subsets, not by repeatedly reusing one value. Input values need not be sorted. Values beyond the target may never be consumed, which is harmless.

```
Empty collection.
The first value is greater than one.
Already complete powers of two.
Duplicate values.
Values much larger than the target.
```

The small oracle enumerates multisets of possible patches and checks subset sums directly.

22.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long target;
    cin >> n >> target;
    vector<long long> values(n);
    for (long long &value : values) cin >> value;
    sort(values.begin(), values.end());
    long long covered = 0;
    int index = 0, patches = 0;
    while (covered < target) {
        if (index < n && values[index] <= covered + 1) {
```

```

        covered += values[index++];
    } else {
        covered += covered + 1;
        ++patches;
    }
}
cout << patches << '\n';
}

```

22.6 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    target = data[1]
    values = sorted(data[2:])
    covered = index = patches = 0
    while covered < target:
        if index < len(values) and values[index] <= covered + 1:
            covered += values[index]
            index += 1
        else:
            covered += covered + 1
            patches += 1
    print(patches)

if __name__ == "__main__":
    main()

```

23 S. Advantage Assignment

23.1 Greedy matching question

Sort both arrays. Consider the smallest value b that has not yet been beaten. Scan values of A from smallest to largest.

- If the current a cannot beat b , it cannot beat any remaining target, because all of them are at least b . It cannot contribute a win.
- If $a > b$, use this smallest sufficient a to beat b . Saving a for a larger target while using an even larger value here cannot create more wins.

This produces a two-pointer algorithm: one pointer scans A , and the other advances through B only after a win.

23.2 Exchange proof

Let a be the smallest remaining value that can beat the smallest unmatched target b . Consider an optimal matching.

If it already matches a with b , it agrees with the greedy choice. Otherwise suppose b is beaten by some $a' \geq a$.

- If a loses or is unused, match a with b and leave a' in its place; the number of wins does not decrease.
- If a beats another target b' , swap the two A values. Since $a' \geq a$ and $a > b'$, a' also beats b' , while a beats b .

Thus an optimal matching exists containing the greedy match. Remove that pair and repeat. If the current a cannot beat b , discarding it is safe because it cannot beat any later target. The final win count is optimal.

23.3 Why sorting both arrays is legitimate

The output asks only for the maximum count, not the actual permutation. Reordering B in the analysis does not change which pairs can be created: any matching between the two multisets can later be placed back at original indices. If the permutation itself were required, original indices would need to travel with B .

23.4 Complexity, pitfalls, and tests

Time is $O(n \log n)$ and memory is $O(n)$.

The inequality is strict: equal values do not win. Pairing the largest values first can waste a very large value on a target that a smaller value could have beaten.

```
All pairs equal -> zero wins.
Every A value wins.
Negative values.
Many duplicates.
One very large A value and several small targets.
```

The oracle tries every permutation of A for small n.

23.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<long long> a(n), b(n);
    for (long long &value : a) cin >> value;
    for (long long &value : b) cin >> value;
    sort(a.begin(), a.end());
    sort(b.begin(), b.end());
    int wins = 0, target = 0;
    for (long long value : a) {
        if (value > b[target]) {
            ++wins;
            ++target;
            if (target == n) break;
        }
    }
    cout << wins << '\n';
}
```

23.6 Full Python solution

```
import sys
```

```
def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    a = sorted(data[1:n + 1])
    b = sorted(data[n + 1:])
    target = wins = 0
    for value in a:
        if value > b[target]:
            target += 1
            wins += 1
            if target == n:
                break
    print(wins)

if __name__ == "__main__":
    main()
```

24 T. Minimum Refueling

24.1 Postpone the decision

When the vehicle passes a reachable station, it does not need to decide immediately whether to stop there. Record that station's fuel as an available past opportunity.

Only when the current reachable distance cannot reach the destination or the next station is a stop forced. At that moment, choose the largest fuel amount among all stations already passed. This extends reach as far as possible while paying exactly one stop.

A maximum heap supports precisely these operations:

```
add the fuel of every newly reachable station;  
when stuck, remove the largest available fuel;
```

If the heap is empty while the destination remains unreachable, the journey is impossible.

24.2 Why a retroactive stop is valid

The algorithm may conceptually decide at position 50 that it should have stopped at position 20. This is valid because position 20 was already reachable before the decision. Treat the heap as a set of optional stops whose fuel could have been collected when they were passed; the order of past stops does not affect their total added fuel.

24.3 Exchange proof

Suppose progress is stuck and the greedy choice takes available station g with maximum fuel. Any solution that reaches farther using one additional available stop chooses some station p with fuel no larger than g . Replacing p by g uses the same number of stops and reaches at least as far.

Therefore an optimal solution exists making the greedy choice whenever another stop becomes necessary. After k greedy stops, no selection of k stations from opportunities available at those decision points can give greater reach. The first time the greedy reach reaches the destination is consequently the minimum possible stop count.

24.4 Sample trace

```

destination = 100, initial reach = 10

reach 10: station (10,60) available -> forced stop, reach 70
reach 70: stations at 20,30,60 enter heap
        choose largest fuel 40 -> reach 110

```

The answer is two. Notice that the stations at 20 and 30 became available without forcing an immediate commitment.

24.5 Complexity, pitfalls, and tests

Sorting and heap operations cost $O(n \log n)$ time. Memory is $O(n)$. Reach and fuel totals require 64-bit integers.

Choosing the next physical station's fuel is not optimal; all passed stations remain alternatives. Do not push a station until its position is reachable. Stopping after the destination has already become reachable adds an unnecessary stop.

```

Destination reachable initially.
No station is initially reachable.
A small nearby station versus a larger earlier option.
Stations supplied out of order.
Reach and fuel near  $10^{18}$ .

```

The oracle enumerates all station subsets and checks them in position order.

24.6 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    long long destination, reachable;
    int n;
    cin >> destination >> reachable >> n;
    vector<pair<long long, long long>> stations(n);
    for (auto &[position, fuel] : stations) cin >> position >> fuel;
    sort(stations.begin(), stations.end());
    priority_queue<long long> available;
    int index = 0, stops = 0;
    while (reachable < destination) {
        while (index < n && stations[index].first <= reachable) {
            available.push(stations[index].second);
            ++index;
        }
    }
}

```

```

        if (available.empty()) {
            cout << -1 << '\n';
            return 0;
        }
        reachable += available.top();
        available.pop();
        ++stops;
    }
    cout << stops << '\n';
}

```

24.7 Full Python solution

```

import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    destination, reachable, n = data[:3]
    stations = sorted(zip(data[3::2], data[4::2]))
    available = []
    index = stops = 0
    while reachable < destination:
        while index < n and stations[index][0] <= reachable:
            heapq.heappush(available, -stations[index][1])
            index += 1
        if not available:
            print(-1)
            return
        reachable -= heapq.heappop(available)
        stops += 1
    print(stops)

if __name__ == "__main__":
    main()

```

25 U. Circular Fuel Start

25.1 Two separate questions

First ask whether any start can work. A complete circuit receives total fuel `sum(fuel)` and spends `sum(cost)`. If the total gain is negative, no starting point can change that fact.

Assume the total gain is nonnegative. Scan stations from left to right while testing a candidate start. Maintain the tank accumulated from that candidate. If it becomes negative after station `i`, reset the candidate to `i + 1`.

The non-obvious part is why this failure eliminates every start between the old candidate and `i`, rather than only the old candidate.

25.2 Failed-block proof

Let the current candidate be `start`. Before the failure at `i`, every proper prefix from `start` had nonnegative gain; otherwise the candidate would already have been reset.

For any `j` between `start + 1` and `i`:

```
gain(j through i)
= gain(start through i) - gain(start through j-1)
```

The first term is negative at the failure, and the second is nonnegative. Therefore the gain from `j` through `i` is negative. Starting at `j` also fails before leaving this block. All candidates through `i` can be discarded at once.

25.3 Why the final candidate completes the circle

After the last reset, the candidate reaches the end of the array without a negative tank. The stations before the candidate form the discarded prefix. Because total gain over the whole circle is nonnegative, the gain over that prefix is enough, when added after the successful suffix, to finish the wrap.

Every smaller index was eliminated by a proved failed block, so the surviving candidate is also the smallest valid index.

25.4 Complexity, pitfalls, and tests

The algorithm is $O(n)$ time and $O(1)$ auxiliary memory. Totals require 64-bit integers.

Checking only total fuel establishes existence, not the start. Returning the index after every locally negative station without accumulating from the current candidate loses the failed-block invariant. Indices are zero-based.

```
One station with equal fuel and cost.  
Negative total gain.  
Several valid starts -> return the smallest.  
Several consecutive resets.  
Large values whose total exceeds 32-bit range.
```

The oracle simulates a full circuit from every possible start.

25.5 Full C++ solution

```
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
    ios::sync_with_stdio(false);  
    cin.tie(nullptr);  
    int n;  
    cin >> n;  
    vector<long long> fuel(n), cost(n);  
    for (long long &value : fuel) cin >> value;  
    for (long long &value : cost) cin >> value;  
    long long total = 0, tank = 0;  
    int candidate = 0;  
    for (int i = 0; i < n; ++i) {  
        long long gain = fuel[i] - cost[i];  
        total += gain;  
        tank += gain;  
        if (tank < 0) {  
            candidate = i + 1;  
            tank = 0;  
        }  
    }  
    cout << (total < 0 ? -1 : candidate) << '\n';  
}
```

25.6 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    fuel = data[1:n + 1]
    cost = data[n + 1:]
    total = tank = candidate = 0
    for i, (received, spent) in enumerate(zip(fuel, cost)):
        gain = received - spent
        total += gain
        tank += gain
        if tank < 0:
            candidate = i + 1
            tank = 0
    print(-1 if total < 0 else candidate)

if __name__ == "__main__":
    main()

```

26 V. Smallest Separated Rearrangement

26.1 Lexicographic construction needs a completion test

At each position, try remaining characters from **a** through **z**, skipping the previous character. Choose the first character after which the remaining multiset can still be arranged without equal neighbors.

The central problem is deriving an exact, fast completion test. Suppose the chosen character is **c** and **R** characters remain.

- Remaining copies of **c** cannot be first, because the prefix now ends in **c**. Each copy needs a different character before it. Therefore $\text{count}[\text{c}] \leq \text{floor}(\text{R}/2)$.
- Any other character may appear first, so its copies need only be separated by the other **R** - $\text{count}[\text{x}]$ characters. Therefore $\text{count}[\text{x}] \leq \text{ceil}(\text{R}/2)$ for every $\text{x} \neq \text{c}$.

These conditions are also sufficient: arrange the most frequent permitted character into alternating slots, with the stricter bound ensuring that the first suffix slot is not forced to equal **c**, then fill the gaps with the remaining characters.

26.2 Making the test fast

Trying at most 26 candidate letters per position is fine, but rescanning all 26 counts for every candidate is unnecessary. For the current count array, build prefix and suffix maxima over the alphabet. They answer the maximum count among all letters except candidate **c** in constant time.

For each candidate:

```
same = count[c] - 1
other = maximum count among letters other than c

same <= R / 2
other <= (R + 1) / 2
```

Take the first candidate satisfying both inequalities.

26.3 Correctness proof

The completion inequalities are necessary by the separator argument above and sufficient by alternating the most frequent letters with the rest. Therefore the test accepts exactly those characters that admit at least one valid suffix.

At each position, the algorithm chooses the smallest character that still admits a completion. Any valid answer beginning with a larger character is lexicographically worse at the first differing position. Choosing the smaller feasible character cannot destroy validity because the completion test supplies a valid suffix. Induction over positions proves the constructed rearrangement is the lexicographically smallest valid one.

26.4 Complexity, pitfalls, and tests

The alphabet has constant size 26. Each position performs $O(26)$ work, so time is $O(26n)$, conventionally $O(n)$, and auxiliary memory is $O(26)$ besides the answer.

The usual condition `maximum frequency <= ceil(length/2)` is insufficient for a suffix whose first character is forbidden; the newly chosen character needs the stricter floor bound. A maximum heap can construct some valid arrangement, but does not automatically make it lexicographically smallest.

```
One character.  
All characters equal.  
Exactly balanced dominant count.  
Several possible answers with different first characters.  
All 26 letters.
```

The oracle performs a memoized lexicographic search over every multiset state.

26.5 Full C++ solution

```
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
    ios::sync_with_stdio(false);  
    cin.tie(nullptr);  
    string s, answer;  
    cin >> s;  
    array<int, 26> count{};  
    for (char c : s) ++count[c - 'a'];  
    int previous = -1;  
    for (int position = 0; position < (int)s.size(); ++position) {  
        array<int, 27> prefix{}, suffix{};  
        for (int c = 0; c < 26; ++c) {  
            prefix[c + 1] = max(prefix[c], count[c]);  
        }  
        for (int c = 25; c >= 0; --c) {  
            suffix[c] = max(suffix[c + 1], count[c]);  
        }  
        int remaining = (int)s.size() - position - 1;  
        int chosen = -1;  
        for (int c = 0; c < 26; ++c) {
```

```

        if (c == previous || count[c] == 0) continue;
        int same = count[c] - 1;
        int other = max(prefix[c], suffix[c + 1]);
        if (same <= remaining / 2
            && other <= (remaining + 1) / 2) {
            chosen = c;
            break;
        }
    }
    if (chosen == -1) {
        cout << "IMPOSSIBLE\n";
        return 0;
    }
    answer.push_back(char('a' + chosen));
    --count[chosen];
    previous = chosen;
}
cout << answer << '\n';
}

```

26.6 Full Python solution

```

import sys

def main() -> None:
    s = sys.stdin.buffer.readline().strip().decode()
    count = [0] * 26
    for char in s:
        count[ord(char) - ord("a")] += 1
    answer = []
    previous = -1
    for position in range(len(s)):
        prefix = [0] * 27
        suffix = [0] * 27
        for char in range(26):
            prefix[char + 1] = max(prefix[char], count[char])
        for char in range(25, -1, -1):
            suffix[char] = max(suffix[char + 1], count[char])
        remaining = len(s) - position - 1
        chosen = -1
        for char in range(26):
            if char == previous or count[char] == 0:
                continue
            same = count[char] - 1
            other = max(prefix[char], suffix[char + 1])
            if same <= remaining // 2 and other <= (remaining + 1) // 2:
                chosen = char
    answer.append(chosen)
    previous = chosen

```

```
        break
    if chosen == -1:
        print("IMPOSSIBLE")
        return
    answer.append(chr(ord("a") + chosen))
    count[chosen] -= 1
    previous = chosen
    print("".join(answer))

if __name__ == "__main__":
    main()
```

27 W. Consecutive Grouping

27.1 Obligations before opportunities

Process values in sorted order. An unfinished group ending at $x - 1$ has only one possible next value: x . If a copy of x exists and such a group is waiting, use x to extend the waiting group before considering a new group.

Maintain:

```
remaining[x] = unused copies of x
need[x]      = unfinished groups whose next required value is x
```

If no group needs x , a copy of x may start a new group only when copies of $x + 1$ and $x + 2$ are immediately available. Reserve all three at once; the new group is now valid with length three and may later be extended by $x + 3$.

27.2 Why extending first is forced

Suppose a group ending at $x - 1$ is waiting, but an algorithm uses x to start another group. The waiting group can never receive a later value: after the sorted scan moves past x , no other copy with the required value appears.

In any valid partition, enough copies of x must first satisfy all existing `need[x]` obligations. Exchanging a copy used by a new group with one used to extend the waiting group cannot harm the new group—it can be started only from copies left after forced obligations are satisfied. Thus extending first is safe and necessary.

27.3 Why starting with three values is safe

When no group needs x , the smallest unused value cannot be appended anywhere. It must start a new group in every valid partition. Such a group must contain $x + 1$ and $x + 2$; if either is unavailable, no valid partition exists. Taking those values immediately commits only choices that every solution must eventually make.

After a new triple is formed, recording a possible future need at $x + 3$ allows it to grow, but its minimum length is already satisfied.

27.4 Complexity, pitfalls, and tests

Sorting costs $O(n \log n)$ time. Map operations are expected $O(1)$ with hash maps, and memory is $O(n)$.

Starting new groups before extending old ones is the classic failure. Checking only that every distinct run has length at least three ignores multiplicities. Negative values require no special treatment.

```
Fewer than three values.  
One exact consecutive triple.  
Two overlapping triples: 1 2 3 3 4 5.  
An unfinished group competing with a new group.  
Duplicates and negative values.
```

The oracle assigns sorted occurrences to every possible existing group or a new group, and accepts only partitions whose final group lengths are at least three.

27.5 Full C++ solution

```
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
    ios::sync_with_stdio(false);  
    cin.tie(nullptr);  
    int n;  
    cin >> n;  
    vector<long long> values(n);  
    unordered_map<long long, int> remaining, need;  
    for (long long &value : values) {  
        cin >> value;  
        ++remaining[value];  
    }  
    sort(values.begin(), values.end());  
    for (long long value : values) {  
        if (remaining[value] == 0) continue;  
        if (need[value] > 0) {  
            --remaining[value];  
            --need[value];  
            ++need[value + 1];  
        } else if (remaining[value + 1] > 0  
                   && remaining[value + 2] > 0) {  
            --remaining[value];  
            --remaining[value + 1];  
            --remaining[value + 2];  
            ++need[value + 3];  
        } else {  
            cout << "NO\n";  
            return 0;  
        }  
    }  
    cout << "YES\n";  
    return 0;  
}
```

```

        return 0;
    }
}
cout << "YES\n";
}

```

27.6 Full Python solution

```

import collections
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    values = sorted(data[1:])
    remaining = collections.Counter(values)
    need = collections.Counter()
    for value in values:
        if remaining[value] == 0:
            continue
        if need[value] > 0:
            remaining[value] -= 1
            need[value] -= 1
            need[value + 1] += 1
        elif remaining[value + 1] > 0 and remaining[value + 2] > 0:
            remaining[value] -= 1
            remaining[value + 1] -= 1
            remaining[value + 2] -= 1
            need[value + 3] += 1
        else:
            print("NO")
            return
    print("YES")

if __name__ == "__main__":
    main()

```

28 X. Subarray Minimum Total

28.1 Start from subarrays ending at one position

Enumerating every subarray repeats almost the same work. A more useful first question is: after reading $a[i]$, what are the minima of all subarrays ending at i ?

If the previous suffix minima, grouped by equal value, are

```
(value, number of suffixes having this minimum),
```

then appending x changes every group with $\text{value} \geq x$ into one group whose minimum is x . Groups with smaller minima remain unchanged. The affected groups form a suffix when the values are stored in increasing order, which is exactly the shape maintained by a monotonic stack.

Maintain `ending_sum`, the sum of minima over all subarrays ending at the previous position. For a new value x , begin with one new suffix $[x]$. Pop every stack group with value at least x , remove its contribution from `ending_sum`, and add its number of suffixes to the new group. Then add the merged contribution $x * \text{ways}$. Add the resulting `ending_sum` to the global answer.

28.2 Why merging equal values is convenient

Using $\geq$ in the pop condition gives every suffix exactly one current group, including duplicates. With 2,2,2, the groups successively have sizes one, two, and three. Using a different strictness convention can also work, but the ownership of equal minima must remain consistent.

28.3 Correctness proof

Before processing x , the stack partitions all suffixes ending at $i-1$ by their minimum. Appending x changes a suffix minimum from m to $\min(m, x)$. Thus precisely the popped groups acquire minimum x ; every unpopped group keeps its smaller minimum. The singleton suffix $[x]$ also has minimum x . Consequently the updated stack partitions every suffix ending at i exactly once, and `ending_sum` is their exact minimum total. Summing `ending_sum` over all right endpoints counts every nonempty subarray exactly once.

28.4 Complexity, pitfalls, and tests

Every group is pushed and popped once, so time is $O(n)$ and memory is $O(n)$. The total requires 64-bit storage in C++. Common mistakes are adding only the new group's contribution without removing popped contributions and handling equal values inconsistently.

```
One value.  
All values equal.  
Strictly increasing and strictly decreasing arrays.  
The minimum appearing several times.
```

The oracle enumerates all left and right endpoints and updates the minimum incrementally.

28.5 Full C++ solution

```
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
    ios::sync_with_stdio(false);  
    cin.tie(nullptr);  
  
    int n;  
    cin >> n;  
  
    // (value, number of suffixes for which value is the minimum)  
    vector<pair<long long, long long>> stack;  
    long long ending_sum = 0;  
    long long answer = 0;  
  
    for (int i = 0; i < n; ++i) {  
        long long value;  
        cin >> value;  
        long long ways = 1;  
  
        while (!stack.empty() && stack.back().first >= value) {  
            auto [old_value, old_ways] = stack.back();  
            stack.pop_back();  
            ending_sum -= old_value * old_ways;  
            ways += old_ways;  
        }  
  
        stack.push_back({value, ways});  
        ending_sum += value * ways;  
        answer += ending_sum;  
    }  
  
    cout << answer << '\n';  
}
```

```
}
```

28.6 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    values = data[1:]

    # Each pair is (minimum, number of suffixes having that minimum).
    stack: list[tuple[int, int]] = []
    ending_sum = 0
    answer = 0

    for value in values[:n]:
        ways = 1
        while stack and stack[-1][0] >= value:
            old_value, old_ways = stack.pop()
            ending_sum -= old_value * old_ways
            ways += old_ways

        stack.append((value, ways))
        ending_sum += value * ways
        answer += ending_sum

    print(answer)

if __name__ == "__main__":
    main()
```

29 Y. Maximum Width Ramp

29.1 Eliminate dominated left endpoints

Suppose $p < q$ and $a[p] \leq a[q]$. Whenever q can pair with some right endpoint j , p can pair with the same j and gives at least as much width. Therefore q is never needed as a left endpoint.

Scanning left to right, retain an index only when its value is a new strict prefix minimum. Their indices increase while their values strictly decrease, so they form a monotonic candidate stack.

29.2 Why scan right endpoints backward

Now inspect j from right to left. If the candidate on top has value at most $a[j]$, then this is the farthest-right valid partner that candidate can ever receive: all later indices were already inspected. Record its width and pop it permanently. Continue while more candidates fit.

A tempting $O(n \log n)$ solution binary-searches the prefix-minimum values for each right endpoint. It is valid, but the reverse scan reveals that both pointers move only once and removes the logarithmic factor.

29.3 Correctness proof

The dominance argument proves that some optimal pair uses a retained prefix-minimum index on the left. Right endpoints are processed in decreasing order, so when a retained index is popped, the current j is its largest possible valid partner and therefore gives its maximum possible width. Every potentially optimal left endpoint is evaluated with its best partner, so the maximum recorded width is optimal.

29.4 Complexity, pitfalls, and tests

Each index enters and leaves the candidate stack at most once. Time is $O(n)$ and memory is $O(n)$. Equal values must not create additional left candidates, but equality is allowed when matching $a[i] \leq a[j]$. Permitting $i = j$ means the answer is always at least zero.

One value.

Strictly increasing and strictly decreasing arrays.

All values equal.

Negative values and repeated prefix minima.

The oracle checks all $O(n^2)$ pairs.

29.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<long long> values(n);
    for (long long& value : values)
        cin >> value;

    // Only new prefix minima can be useful left endpoints.
    vector<int> candidates;
    for (int i = 0; i < n; ++i)
        if (candidates.empty() || values[i] < values[candidates.back()])
            candidates.push_back(i);

    int answer = 0;
    for (int j = n - 1; j >= 0; --j) {
        while (!candidates.empty() &&
               values[candidates.back()] <= values[j]) {
            answer = max(answer, j - candidates.back());
            candidates.pop_back();
        }
    }

    cout << answer << '\n';
}
```

29.6 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    values = data[1:]

    # Indices of strict prefix minima: all useful left endpoints.
    candidates: list[int] = []
    for i in range(n):
        if not candidates or values[i] < values[candidates[-1]]:
```

```
        candidates.append(i)

    answer = 0
    for j in range(n - 1, -1, -1):
        while candidates and values[candidates[-1]] <= values[j]:
            answer = max(answer, j - candidates.pop())

    print(answer)

if __name__ == "__main__":
    main()
```

30 Z. Smallest Alternating Partition

30.1 Separate feasibility from label choice

Process positions from left to right. A bit 0 can extend only a group ending in 1, and conversely. If such a group exists, reuse one. If none exists, the current bit cannot belong to any existing group, so creating a new group is forced.

Why is reuse safe when starting a new group might preserve the opposite-ending group for later? If that preserved group is used by a later equal bit, swap the two suffix assignments: let the current bit extend the old group and let the later bit take the newly created group's role. Alternation and the final number of groups are unchanged. Thus an optimum exists that reuses a group now.

This proves the primary objective. For the secondary objective, all groups ending in the required opposite bit are equivalent for future feasibility. Choose the smallest available label, because it makes the first currently undecided output position as small as possible. Store the available labels for each ending bit in a minimum heap.

30.2 Correctness proof

By induction over positions, an optimal partition exists that agrees with all greedy reuse-or-create decisions. Creation is forced when no group has the opposite ending; otherwise the suffix-swap argument makes reuse compatible with an optimum. Hence the final number of groups is minimum.

Among all minimum-group completions of the established prefix, choosing the smallest compatible label produces the smallest value at the next output position. Compatible groups have identical future behavior apart from their labels, so exchanging their future roles preserves feasibility. Induction then proves the complete label sequence is lexicographically smallest.

30.3 Complexity, pitfalls, and tests

Each position performs a constant number of heap operations, so time is $O(n \log n)$ and memory is $O(n)$. A stack is sufficient to minimize the number of groups but not to guarantee the smallest label sequence. New labels must be created consecutively to respect the canonical-label convention.

```
One bit.  
All bits equal.  
An already alternating string.  
011, where creating early can still tie the minimum but loses lexicographically.  
Several compatible groups with different labels.
```

The oracle backtracks over canonical assignments and compares first by group count, then by the complete label tuple.

30.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    cin >> s;

    priority_queue<int, vector<int>, greater<int>> ending[2];
    vector<int> answer;
    int groups = 0;

    for (char character : s) {
        int bit = character - '0';
        int opposite = 1 - bit;
        int group;

        if (!ending[opposite].empty()) {
            group = ending[opposite].top();
            ending[opposite].pop();
        } else {
            group = ++groups;
        }

        ending[bit].push(group);
        answer.push_back(group);
    }

    cout << groups << '\n';
    for (int i = 0; i < static_cast<int>(answer.size()); ++i)
        cout << answer[i] << " \n"[i + 1 == static_cast<int>(answer.size())];
}
```

30.5 Full Python solution

```
import heapq
import sys
```

```

def main() -> None:
    s = sys.stdin.buffer.readline().strip().decode()
    ending = [[], []]
    answer = []
    groups = 0

    for character in s:
        bit = int(character)
        opposite = 1 - bit
        if ending[opposite]:
            group = heapq.heappop(ending[opposite])
        else:
            groups += 1
            group = groups
        heapq.heappush(ending[bit], group)
        answer.append(group)

    print(groups)
    print(*answer)

if __name__ == "__main__":
    main()

```